

AST-LLM: Adaptive Sparse Training for Energy-Efficient Large Language Models

Wei Chen¹, Ming Zhang^{1,2}, Jing Liu¹, Hao Wang²

¹School of Computer Science, Beijing University ²AI Research Lab, Tsinghua University

Submitted to NeurIPS 2026

Abstract

The rapid scaling of Large Language Models (LLMs) has driven unprecedented gains in natural language understanding, but at the cost of enormous computational and energy requirements. Sparse training offers a promising avenue for reducing these costs by updating only a subset of parameters. However, existing methods rely on static sparsity patterns that fail to adapt to the evolving loss landscape. We propose AST-LLM (Adaptive Sparse Training for LLMs), a framework that dynamically adjusts sparsity patterns based on gradient signal importance. AST-LLM introduces three key innovations: (1) a gradient-aware sparsity allocation mechanism (GASA) that redistributes non-zero parameters across layers according to their contribution to loss reduction; (2) a hysteresis pruning schedule (HPS) that prevents premature removal of potentially important connections; and (3) a hardware-aware block-sparse format achieving 3.2× wall-clock speedup on NVIDIA A100 GPUs. Evaluated on Llama-2 (7B, 13B) and GPT-3-style (1.3B, 6.7B) architectures trained on the Pile and RefinedWeb, AST-LLM achieves 90% sparsity with only 0.8% perplexity degradation, reduces training FLOPs by 4.7×, and lowers energy consumption by 3.9×. Ablation studies reveal GASA contributes 62% of the total improvement.

1. Introduction

Large Language Models have transformed AI, achieving remarkable performance across text generation, translation, summarization, and code synthesis [1-4]. The dominant paradigm has been to scale model size, data, and compute [5,6]. GPT-4 contains over 1.7 trillion parameters; PaLM-2 and Llama-3 push boundaries further. However, this scaling comes at staggering cost: training GPT-4 reportedly consumed over 50 GWh of electricity, equivalent to the annual consumption of 5,000 US households [7].

The environmental and economic implications have sparked intense interest in efficient training. Among the most promising approaches is sparse training, where only a fraction of parameters are updated per iteration [8-10]. By maintaining dynamic sparse connectivity, sparse training can reduce both memory footprint and computational cost proportionally to sparsity ratio. Early work focused on vision tasks with CNNs [11,12], demonstrating 80-90% of weights could be pruned without significant accuracy loss. Recent efforts extend these techniques to transformers [13,14], but face fundamental challenges unique to LLMs.

The key challenge: the LLM loss landscape is highly non-stationary. Optimal sparsity at initialization differs dramatically from the pattern needed during convergence. Static masks—whether set at initialization (SNIP [15], GraSP [16]) or periodically recomputed (RigL [17], SET [18])—cannot capture nuanced gradient dynamics across billions of parameters. Furthermore, load imbalance from unstructured sparsity leads to poor hardware utilization, often negating theoretical FLOPs reduction [19,20].

We propose AST-LLM, jointly optimizing sparsity pattern, training dynamics, and hardware efficiency. Key contributions: (1) gradient-aware sparsity allocation (GASA) continuously redistributing the sparsity budget across layers; (2) hysteresis pruning schedule (HPS) using dual-threshold to prevent oscillatory pruning; (3) block-sparse implementation achieving 3.2× speedup. AST-LLM achieves 90% sparsity with negligible quality loss on models up to 13B parameters.

2. Related Work

2.1 Sparse Neural Network Training

Sparse training methods divide into static and dynamic approaches. Static methods fix the sparsity pattern before training, using criteria such as weight magnitude [21], gradient signal [15,16], or random initialization [22]. While efficient, they cannot adapt to changing gradient landscapes. Dynamic methods periodically update the pattern. SET [18] randomly regrows pruned connections every ΔT steps. RigL [17] improves SET by regrowing based on gradient magnitude. DSR [23] introduced learnable pruning thresholds via straight-through estimation. However, these methods were designed for moderate-scale vision models and struggle with LLM-scale training due to mask update overhead and lack of layer-wise adaptation.

2.2 Efficient Transformer Training

Several lines of work target efficient transformer training: mixed-precision [24] reduces memory bandwidth; gradient checkpointing [25] trades compute for memory; FlashAttention [26,27] restructures attention to minimize HBM accesses; ZeRO [28] and FSDP [29] partition states across workers. While complementary, these methods do not reduce fundamental parameter count. Combining AST-LLM with these yields multiplicative gains.

2.3 Lottery Ticket Hypothesis at Scale

The Lottery Ticket Hypothesis (LTH) [30] showed dense networks contain sparse subnetworks trainable in isolation to match full accuracy. Subsequent work extended LTH to larger models [31] and transformers [32,33]. Super Tickets [34] and Prune Once for All [35] simplified the pipeline. However, LTH methods require multiple rounds of training, impractical for LLMs. We draw inspiration from LTH while enabling single-pass training for billion-parameter models.

3. AST-LLM Framework

AST-LLM consists of three integrated components enabling efficient adaptive sparse training. Figure 1 provides an architectural overview.

Figure 1. Architecture of AST-LLM. (A) GASA module computes per-layer importance from gradient statistics and redistributes the global sparsity budget. (B) HPS uses dual thresholds for pruning/regrowth decisions. (C) Block-sparse format for efficient GPU execution.

3.1 Gradient-Aware Sparsity Allocation (GASA)

Traditional methods apply uniform sparsity across layers, but transformer layers exhibit vastly different pruning sensitivity. Early embedding layers and final LM heads are particularly sensitive, while intermediate FFN layers in deeper blocks tolerate much higher sparsity. GASA dynamically allocates the sparsity budget.

Formally, for L layers, global target sparsity S , and mean gradient magnitude $g_l(t)$ at step t , the allocation for layer l is:

$$s_l(t) = S * (g_l(t)^\alpha) / (\sum_i g_i(t)^\alpha)$$

where α is a temperature hyperparameter. $\alpha=0$ gives uniform allocation; larger α concentrates non-zero parameters in high-gradient layers. We found $\alpha=0.5$ optimal across model scales. EMA smoothing with $\beta=0.99$ prevents oscillation from mini-batch gradient noise.

3.2 Hysteresis Pruning Schedule (HPS)

Dynamic sparse training suffers from "flip-flopping": weights near the pruning threshold oscillate between pruned and regrown states, wasting resources. HPS uses dual thresholds inspired by physical hysteresis.

HPS maintains pruning threshold θ_p and regrowth threshold θ_r ($\theta_r > \theta_p$). A weight is removed only when $|w| < \theta_p$ for k consecutive cycles. A missing connection regrows only when gradient magnitude $|\nabla_w| > \theta_r$ for m consecutive cycles. The hysteresis gap $\Delta = \theta_r - \theta_p$ adapts based on layer-wise gradient variance: $\Delta_l = \Delta_0 * \sigma_l / \sigma_{\text{mean}}$, where σ_l is gradient std of layer l , σ_{mean} is mean across layers. $\Delta_0 = 0.15$ provides robust performance.

3.3 Hardware-Aware Block-Sparse Implementation

Unstructured sparsity leads to irregular memory access that modern GPUs handle poorly. We enforce sparsity at 32×32 contiguous blocks, enabling coalesced memory access and efficient Tensor Core utilization. Our kernel achieves $3.2\times$ speedup over dense GEMM at 90% sparsity (vs $1.3\times$ for unstructured). The two-level index structure (block-level CSR mapping to dense sub-matrices) enables direct Tensor Core loading without explicit gather/scatter.

3.4 Training Pipeline

AST-LLM integrates into standard training with minimal overhead. The sparsity mask updates every $T_{\text{update}} = 500$ steps ($< 0.3\%$ of total training time). Algorithm 1 summarizes the procedure.

Algorithm 1. AST-LLM Training: (1) Initialize dense model; (2) Per step: forward/backward with current mask; (3) Every T_{update} steps: update gradient EMA, recompute layer-wise allocation via GASA, update masks via HPS; (4) Continue to convergence.

4. Experimental Setup

4.1 Models and Datasets

We evaluate on four configurations: GPT-3-style (1.3B, 6.7B) and Llama-2 (7B, 13B) with standard transformer architecture (pre-normalization, RoPE, SwiGLU). Training data: the Pile (825 GB, 22 subdomains) and RefinedWeb (5T tokens). We also test on CodeParrot and PubMed subsets for domain-specific evaluation.

4.2 Baselines

We compare against: Dense training, Static magnitude pruning (MP), SNIP [15], SET [18], RigL [17], and DSR [23]. All sparse methods target 90% global sparsity unless specified.

4.3 Hyperparameters

AdamW optimizer ($\beta_1=0.9$, $\beta_2=0.95$), weight decay 0.1, gradient clipping 1.0. Cosine LR schedule, 2000 warmup steps, peak LR $3e-4$ (1.3B/6.7B) or $2e-4$ (7B/13B). Batch size 512-2048 tokens. 64-256 A100-80GB GPUs, FSDP with bfloat16.

5. Results

5.1 Main Results

Table 1 summarizes results across all scales. AST-LLM achieves 90% sparsity with only 0.8% average perplexity degradation, significantly outperforming RigL (2.3%), SET (3.7%), and MP (6.1%). Training FLOPs reduction of 4.7× approaches the theoretical maximum (10× at 90% sparsity), demonstrating excellent hardware utilization.

	PPL-Wiki↓	PPL-C4↓	FLOPs↓	Time↓
Model A	66.1	73.4	84.0	65.1
Model B	78.4	66.1	83.2	94.2
Model C	91.9	88.1	5.5	4.0
Model D	76.6	79.8	90.7	6.8
Model E	84.7	90.7	2.8	8.1

Table 1: Main results. Metrics: Perplexity (WikiText-103, C4), training FLOPs (relative), wall-clock time (relative).

5.2 Scaling Behavior

Figure 2 shows perplexity degradation vs model size. The relative benefit of AST-LLM increases with scale: for 1.3B, AST-LLM outperforms RigL by 0.9 PPL; for 13B, the gap widens to 2.4 PPL. This suggests adaptive sparsity becomes increasingly important at larger scales where loss landscapes are more heterogeneous.

Figure 2. Scaling behavior. (A) WikiText-103 PPL vs model size. (B) C4 PPL vs model size. (C) Training FLOPs reduction ratio. (D) Memory usage comparison across methods.

We hypothesize this scaling advantage stems from: (1) larger models exhibit greater heterogeneity in gradient statistics, providing more opportunity for GASA; (2) HPS is more effective in deeper networks where gradient noise accumulates.

5.3 Ablation Study

Figure 3 presents ablation isolating each component on Llama-2 7B at 90% sparsity. Starting from SET baseline, we incrementally add gradient-based regrowth ($\rightarrow$ RigL), GASA, HPS, and block-sparse implementation.

Figure 3. Ablation on Llama-2 7B. (A) WikiText-103 PPL. (B) C4 PPL. (C) Training FLOPs. (D) GASA α sweep. (E) HPS $\Delta\blacksquare$ sweep. (F) T_{update} sweep.

GASA contributes 62% of total perplexity improvement—confirming adaptive allocation is the most impactful innovation. HPS contributes 28% by reducing late-stage training instability. Block-sparse implementation contributes 10% to practical speedup with negligible quality impact. Hyperparameter sweeps confirm $\alpha=0.5$, $\Delta\blacksquare=0.15$, $T_{\text{update}}=500$ are robust across scales.

5.4 Layer-wise Sparsity Distribution

Figure 4 visualizes the GASA-learned sparsity distribution for Llama-2 7B. The pattern is highly non-uniform: embedding layers at 65-75% sparsity, early transformer blocks at 88-92%, middle blocks at 92-95%, late blocks at 85-90%, and LM head at only 55% sparsity, confirming its critical role. This validates GASA's core premise.

Figure 4. Learned layer-wise sparsity for Llama-2 7B. (A) Sparsity ratio per layer. (B) Gradient magnitude vs assigned sparsity. (C) Evolution of sparsity distribution over training. (D) Attention head importance heatmap.

5.5 Energy Efficiency

Table 2 reports actual energy consumption using NVIDIA-smi monitoring. AST-LLM reduces energy by 3.9× for 13B training, saving ~42 MWh—equivalent to 4 households annual electricity. Carbon footprint uses US average grid intensity (0.385 kg CO₂/kWh).

	Energy(kWh)↓	CO ₂ (kg)↓	Speedup×
Model A	78.0	92.7	65.8
Model B	5.3	5.9	7.0
Model C	73.3	93.2	87.6
Model D	66.6	76.6	66.8

Table 2: Energy consumption and carbon footprint at 90% sparsity.

5.6 Structured Pruning Comparison

AST-LLM significantly outperforms structured pruning methods. Head pruning (removing 90% of attention heads) degrades PPL by 5.8%, FFN dimension pruning by 4.2%, layer dropping by 7.3%, while AST-LLM degrades by only 0.8%. Structured methods impose rigid constraints, discarding useful partial information.

	PPL↓	MMLU	Speedup
Model A	76.2	65.2	78.1
Model B	77.7	89.6	6.2
Model C	93.1	4.8	8.2
Model D	6.4	67.4	89.4
Model E	77.9	91.2	82.5
Model F	6.0	69.9	78.5

Table 3: Comparison with structured pruning on Llama-2 7B.

5.7 Downstream Tasks

Beyond perplexity, we evaluate on MMLU (57-subject accuracy), HellaSwag (commonsense), ARC-Challenge (science reasoning), GSM8K (math). AST-LLM at 90% sparsity matches or exceeds dense baseline on 3/4 benchmarks, with only 1.2% MMLU drop. Sparse training preserves both language modeling and reasoning capabilities.

	MMLU↑	HellaSwag↑	ARC-C↑	GSM8K↑
Model A	1.7	75.5	5.9	80.4
Model B	84.0	3.5	6.0	70.8
Model C	74.8	7.5	93.9	5.5
Model D	74.5	94.6	5.0	78.1
Model E	87.5	70.6	2.9	80.1

Table 4: Downstream performance on Llama-2 7B at 90% sparsity.

5.8 Training Dynamics

Figure 5 analyzes training dynamics. AST-LLM exhibits three-phase convergence: (1) rapid pruning (0-5k steps) where the mask stabilizes; (2) exploration (5k-20k) where GASA reallocates budget; (3) convergence (>20k) where both weights and mask converge. RigL and SET show persistent mask fluctuations throughout, indicating instability.

Figure 5. Training dynamics. (A) PPL vs steps. (B) Mask change rate (Jaccard). (C) Gradient norm across layers. (D) PCA projection of loss landscape. (E) Effective rank of weight matrices.

5.9 Few-shot Evaluation

We evaluate few-shot performance on Llama-2 7B with 0-shot, 1-shot, and 5-shot settings across 10 diverse tasks. AST-LLM retains 97% of dense few-shot accuracy at 90% sparsity, compared to 89% for RigL. The gap is largest for knowledge-intensive tasks (MMLU, TriviaQA) where preserving

dense connections in critical layers matters most.

	MMLU	TriviaQA	ARC-C
Model A	70.9	75.6	68.1
Model B	70.5	84.5	75.4
Model C	74.6	77.5	83.0
Model D	86.2	65.6	76.1
Model E	8.4	91.6	76.8

Table 5: Few-shot evaluation (Llama-2 7B, 5-shot).

6. Analysis and Discussion

6.1 Why Adaptive Sparsity Helps More at Scale

The benefit of adaptive sparsity increases with model size due to growing heterogeneity of layer-wise loss landscapes. In 1.3B models, gradient magnitude variance across layers is $\sim 2.3\times$; in 13B, it grows to $6.8\times$. GASA exploits this by concentrating parameters where gradients are strongest. This has important implications: as models scale, adaptive approaches become necessities rather than luxuries.

6.2 Gradient Flow Analysis

We analyze gradient flow through sparse masks using the effective gradient norm ratio—the ratio of gradient norm reaching early layers relative to the final layer. Dense training maintains 0.72 ratio; AST-LLM maintains 0.68; RigL drops to 0.41; SET to 0.28. This indicates AST-LLM better preserves gradient signal propagation, crucial for training deep transformers.

6.3 Compute-Memory Tradeoffs

AST-LLM introduces a modest memory overhead (12% for storing dual thresholds and gradient statistics) but achieves significant compute savings ($4.7\times$ FLOPs reduction). For GPU-hour-constrained researchers, this represents a favorable tradeoff. The block-sparse format adds 5% memory overhead for index structures but enables $3.2\times$ practical speedup.

6.4 Robustness to Hyperparameter Variation

We conducted extensive sensitivity analysis across key hyperparameters. AST-LLM is robust to α in $[0.3, 0.7]$, Δ in $[0.10, 0.20]$, and T_{update} in $[200, 1000]$. Performance degrades gracefully outside these ranges: setting α too high (>1.0) concentrates parameters excessively in few layers, while α too low (<0.1) approximates uniform allocation and loses adaptive benefit.

6.5 Comparison with Knowledge Distillation

We compare AST-LLM with knowledge distillation (KD), where a sparse student learns from a dense teacher. At 90% sparsity, KD achieves 1.5% PPL degradation (vs 0.8% for AST-LLM) but requires the additional cost of training the teacher model. AST-LLM provides a self-contained solution without teacher overhead, making it more practical for scenarios where a pre-trained dense model is unavailable.

6.6 Convergence Guarantees

We provide theoretical analysis of convergence for AST-LLM under standard assumptions (L-smooth loss, bounded gradient variance). With the hysteresis schedule, the effective learning rate for each parameter is modulated by the mask state, creating an implicit annealing effect. Under mild conditions, AST-LLM converges to a stationary point at rate $O(1/\sqrt{T})$, matching dense training convergence rate.

5. Additional Experimental Results

5.10 Multilingual Evaluation

We evaluate on multilingual benchmarks: XNLI (15 languages), Flores-200 (translation quality), and MMLU-multilingual. AST-LLM maintains 96% of dense multilingual accuracy, with no language disproportionately affected. This suggests adaptive sparsity preserves the multilingual representations learned during pre-training.

	XNLI↑	Flores↑	MMLU-M↑
Model A	90.6	3.6	8.1
Model B	86.2	73.0	74.7
Model C	74.4	2.1	3.7
Model D	6.1	85.5	70.7
Model E	74.9	77.0	93.5

Table 6: Multilingual evaluation on Llama-2 7B. XNLI accuracy averaged across 15 languages.

5.11 Long Sequence Performance

We test on long-range tasks: PG-19 (books, avg 70K tokens), SCROLLS (summarization), and LongBench. AST-LLM maintains 95% of dense performance at 32K context length. At 64K+ tokens, performance gap narrows as attention computation dominates over FFN computation.

Figure 6. Long sequence evaluation. (A) PPL vs sequence length on PG-19. (B) ROUGE-L on SCROLLS. (C) LongBench composite score. (D) Attention vs FFN compute ratio at different lengths.

5.12 Robustness to Initialization

We test AST-LLM across 5 different random initializations. The variance in final perplexity is 0.12 (vs 0.08 for dense), indicating good reproducibility. The learned sparsity patterns across different seeds show 78% Jaccard similarity in layer-wise allocation, suggesting GASA converges to a stable, non-random allocation.

	PPL($\mu \pm \sigma$)	FLOPs↓	MaskSim↑
Model A	94.0	93.4	65.0
Model B	6.5	4.3	78.4
Model C	75.4	67.0	69.0

Table 7: Robustness across 5 random seeds on Llama-2 7B. Mean $\pm$ std reported.

5.13 Inference Efficiency

While AST-LLM primarily targets training efficiency, we also evaluate inference. The AST-LLM-trained sparse model achieves 2.8 $\times$ inference speedup with block-sparse kernels, and can be further compressed via structured pruning for 4.1 $\times$ speedup with 2.3% additional PPL degradation. This dual-use capability—efficient training producing efficient models—is a practical advantage.

	Tokens/s↑	Latency↓	PPL	Speedup×
Model A	82.6	1.8	65.8	7.6
Model B	88.9	78.4	91.8	79.3
Model C	65.4	74.4	5.1	7.9
Model D	82.3	73.6	89.1	6.7

Table 8: Inference efficiency on A100. Throughput (tokens/sec), latency (ms/token).

2.5 Preliminaries: Sparse Training Formalism

We establish formal notation. Let $f(x; \theta)$ be a neural network with parameters $\theta \in \mathbb{R}^d$. In sparse training, we maintain a binary mask $m \in \{0,1\}^d$ where $m_i=1$ indicates an active parameter. The effective parameters are $\theta^\# = \theta \odot m$. The training objective is $\min_{\theta} L(f(x; \theta^\#), y)$ subject to $\|m\|_1 \leq (1-S) \cdot d$, where S is the target sparsity. Dynamic sparse training updates both θ and m : $\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \nabla_{\theta} L(\theta^\#, m^{(t)})$, and $m^{(t+1)} = \text{UpdateMask}(\theta^{(t)}, \nabla_{\theta} L(\theta^\#, m^{(t)}))$. The mask update function is the key differentiator across methods.

AST-LLM extends this formalism with two key generalizations: (1) layer-wise sparsity budgets S_l replace the global constraint; (2) the mask update uses a stateful hysteresis mechanism rather than a stateless threshold. The combination enables both spatial adaptation (which layers get parameters) and temporal stability (preventing oscillatory mask changes).

3.5 Initialization and Warmup Strategy

Proper initialization is critical for sparse training. We initialize AST-LLM with a warmup phase (first 2000 steps) at 50% sparsity using uniform allocation. After warmup, GASA activates and the sparsity linearly increases to the target 90% over the next 3000 steps. This graduated approach prevents catastrophic pruning of essential early-stage connections. The warmup phase adds negligible overhead (<1% of total training) while significantly improving final model quality (0.4 PPL improvement vs cold start).

3.6 Gradient Accumulation and Micro-Batching

LLM training typically uses gradient accumulation across micro-batches. AST-LLM computes gradient statistics on accumulated gradients rather than per-micro-batch, reducing noise in the GASA allocation. For a global batch of 2048 tokens split across 16 micro-batches of 128, gradient statistics are aggregated before computing layer-wise sparsity allocation. This adds negligible overhead since statistics are only computed every T_{update} steps.

3.7 Distributed Training Considerations

In the FSDP setting, each GPU holds a shard of parameters. AST-LLM computes layer-wise gradient statistics using local all-reduce operations. The communication overhead is $O(L)$ per mask update (L layers), which is negligible compared to gradient synchronization. For the block-sparse format, we ensure block boundaries align with FSDP shard boundaries to minimize cross-device communication during sparse GEMM.

Appendix A: Extended Hyperparameter Analysis

Table A1 provides the complete hyperparameter sweep for GASA α across all model scales. The optimal α shifts slightly with model size: 0.45 for 1.3B, 0.50 for 6.7B/7B, 0.55 for 13B. This suggests larger models benefit from slightly more concentrated allocation, consistent with their greater layer-wise heterogeneity.

	1.3B	6.7B	7B	13B
Model A	83.8	91.8	1.6	3.8
Model B	1.7	74.9	1.7	7.6
Model C	87.2	73.9	66.5	77.8
Model D	83.9	3.4	81.7	5.5
Model E	94.7	85.6	87.7	79.0

Table A1: GASA α sweep across model scales. WikiText-103 PPL reported.

Table A2 shows the HPS Δ sweep. Performance is stable across $\Delta \in [0.10, 0.20]$ with degradation outside this range. Too small Δ (<0.05) causes excessive flip-flopping; too large (>0.30) makes the mask too rigid to adapt.

	PPL↓	FlipRate↓	Stable?
Model A	78.1	73.3	74.8
Model B	6.4	74.3	86.3
Model C	82.1	68.6	6.8
Model D	65.4	76.8	70.6

Table A2: HPS Δ sweep on Llama-2 7B. PPL and mask flip rate reported.

Appendix B: Detailed Per-Layer Analysis

We provide the complete per-layer sparsity distribution for all 32 transformer blocks, embedding layer, and LM head. Table B1 reports the final sparsity ratio, gradient magnitude, and effective parameter count for each layer. The attention output projection layers consistently show the highest sensitivity, likely due to their role in aggregating multi-head information.

	Sparsity%	$ \nabla $	ActiveParams
Model A	93.3	3.9	83.0
Model B	72.2	94.6	79.9
Model C	86.4	90.4	93.9
Model D	65.2	92.0	1.8
Model E	68.1	81.0	2.0
Model F	94.0	66.9	5.5
Model G	67.9	65.2	2.9
Model H	83.1	7.3	1.9
Model I	90.8	79.0	83.0

Model J	84.8	67.4	75.9
Model K	6.2	69.8	6.1
Model L	84.9	80.1	6.6
Model M	94.8	74.8	80.5
Model N	91.9	91.6	3.7
Model O	94.9	77.5	84.0
Model P	1.6	7.1	93.2
Model Q	3.3	89.6	8.2
Model R	67.7	73.7	94.3
Model S	87.9	2.9	91.6
Model T	85.8	5.5	84.8
Model U	78.2	92.3	5.7
Model V	76.2	91.1	74.2
Model W	67.4	91.3	7.2
Model X	1.6	92.8	7.8
Model Y	66.6	7.9	72.9
Model Z	76.2	6.3	85.7
Model [	75.5	4.4	88.8
Model \	4.1	5.7	3.2
Model]	87.4	90.3	78.2
Model ^	86.6	3.6	6.2
Model _	84.3	69.5	88.4
Model `	88.9	82.1	71.0
Model a	4.6	66.6	88.6

Table B1: Per-layer sparsity for Llama-2 7B (32 layers).

Appendix C: Compute Infrastructure Details

All experiments were conducted on an internal cluster with 256 NVIDIA A100-80GB GPUs interconnected via InfiniBand HDR (200 Gbps). Training the 13B model with AST-LLM required 128 GPUs for 48 hours. Dense training baseline required 256 GPUs for 96 hours. Carbon footprint estimated using CodeCarbon with real-time grid intensity monitoring. All experiments used PyTorch 2.1 with CUDA 12.1. We will release code, trained masks, and experiment configurations upon publication.

Table C1 details the hardware configuration and training time for each experiment.

	GPUs	Hours	kWh	CO ₂ (kg)
Model A	65.4	85.7	4.1	75.4
Model B	88.2	88.5	4.0	70.2
Model C	93.0	5.7	77.5	68.5
Model D	89.1	73.8	73.6	5.2

Table C1: Compute infrastructure and training duration for each experiment.

Appendix D: Additional Qualitative Examples

We provide qualitative examples comparing AST-LLM and dense model outputs on standard prompts. Table D1 shows generation samples across diverse tasks. AST-LLM generations are nearly indistinguishable from dense baselines, consistent with the quantitative metrics.

	AST-LLM Win%	Dense Win%	Tie%
Model A	74.7	66.5	88.3
Model B	76.5	6.6	89.8
Model C	81.6	73.2	5.8
Model D	91.3	85.8	4.2

Table D1: Generation quality comparison on Llama-2 7B. Human preference win rate (vs dense).

7. Limitations and Future Work

AST-LLM has several limitations. First, optimization is A100-specific; other hardware (TPUs, AMD GPUs) may not achieve the same speedup. Second, GASA assumes gradient magnitude reliably proxies parameter importance—this may not hold universally. Third, we only validated decoder-only architectures; encoder-decoder (T5) and MoE models remain untested. Fourth, block-sparse format limits maximum sparsity to ~95% before block-level artifacts appear.

Future work: (1) extending to mixture-of-experts architectures where dynamic expert selection synergizes with dynamic sparsity; (2) developing hardware-agnostic block-sparse kernels via Triton/OpenAI compiler; (3) exploring AST-LLM for continual pre-training and domain adaptation; (4) integrating with quantization for further efficiency gains; (5) applying GASA principles to structured sparsity for inference optimization.

8. Broader Impact

The environmental impact of LLM training is pressing. By reducing training energy 3.9×, AST-LLM represents a concrete step toward sustainable AI. If adopted for next-generation frontier models, it could prevent tens of thousands of tons of CO₂ emissions. However, efficiency improvements risk Jevons paradox—by making training cheaper, it may incentivize even larger models. Responsible deployment should couple with institutional compute budgeting and carbon accounting policies.

On accessibility: reducing LLM training cost by 4.7× could democratize LLM research, enabling academic labs and institutions in developing countries to participate in foundation model development. This aligns with broader goals of equitable AI research.

9. Conclusion

We presented AST-LLM, an adaptive sparse training framework enabling efficient LLM training at 90% sparsity with negligible quality loss. Three components—GASA, HPS, and block-sparse implementation—collectively achieve $4.7\times$ FLOPs reduction and $3.9\times$ energy savings. Comprehensive evaluation across four model scales demonstrates consistent improvement over existing methods with increasing advantage at larger scales. AST-LLM represents a step toward economically and environmentally sustainable AI, and we hope it inspires further research at the intersection of efficient ML and large-scale systems.

References

- [1] Brown et al. "Language Models are Few-Shot Learners." NeurIPS 2020.
- [2] Touvron et al. "Llama 2: Open Foundation and Fine-Tuned Chat Models." arXiv 2023.
- [3] OpenAI. "GPT-4 Technical Report." arXiv 2023.
- [4] Anil et al. "PaLM 2 Technical Report." arXiv 2023.
- [5] Kaplan et al. "Scaling Laws for Neural Language Models." arXiv 2020.
- [6] Hoffmann et al. "Training Compute-Optimal Large Language Models." NeurIPS 2022.
- [7] Patterson et al. "Carbon Emissions and Large Neural Network Training." arXiv 2021.
- [8] Hoefler et al. "Sparsity in Deep Learning." JMLR 2021.
- [9] Gale et al. "The State of Sparsity in Deep Neural Networks." arXiv 2019.
- [10] Blalock et al. "What is the State of Neural Network Pruning?" MLSys 2020.
- [11] Han et al. "Learning both Weights and Connections." NeurIPS 2015.
- [12] Frankle & Carbin. "The Lottery Ticket Hypothesis." ICLR 2019.
- [13] Chen et al. "The Lottery Ticket Hypothesis for Pre-trained BERT." NeurIPS 2020.
- [14] Kurtic et al. "The Optimal BERT Surgeon." ICML 2022.
- [15] Lee et al. "SNIP: Single-shot Network Pruning." ICLR 2019.
- [16] Wang et al. "GraSP: Gradient Signal Preservation." ICLR 2020.
- [17] Evci et al. "Rigging the Lottery: Making All Tickets Winners." ICML 2020.
- [18] Mocanu et al. "Scalable Training with Adaptive Sparse Connectivity." Nature Comm 2018.
- [19] Yao et al. "Efficient Deep Learning: A Survey." ACM CSUR 2023.
- [20] Liu et al. "Sparse Training: A Comprehensive Survey." arXiv 2024.
- [21] Janowsky. "Pruning versus Clipping." Physical Review E 1989.
- [22] Liu et al. "Rethinking the Value of Network Pruning." ICLR 2019.
- [23] Kukupati et al. "DSR: Differentiable Sparsity." ICML 2021.
- [24] Micikevicius et al. "Mixed Precision Training." ICLR 2018.
- [25] Chen et al. "Training Deep Nets with Sublinear Memory Cost." arXiv 2016.
- [26] Dao et al. "FlashAttention." NeurIPS 2022.
- [27] Dao. "FlashAttention-2." arXiv 2023.
- [28] Rajbhandari et al. "ZeRO: Memory Optimizations." SC 2020.
- [29] Zhao et al. "PyTorch FSDP." VLDB 2023.
- [30] Frankle & Carbin. "The Lottery Ticket Hypothesis." ICLR 2019.
- [31] Renda et al. "Comparing Rewinding and Fine-tuning." ICLR 2020.
- [32] Chen et al. "Lottery Ticket for Pre-trained BERT." NeurIPS 2020.
- [33] Prasanna et al. "When BERT Plays the Lottery." EMNLP 2020.
- [34] Zhou et al. "Deconstructing Lottery Tickets." NeurIPS 2019.
- [35] Michel et al. "Prune Once for All." arXiv 2021.